

TECHNICAL BUILD PLAN & ARCHITECTURE

TelConnect Build Plan & Implementation Architecture (v7.0)

Technical Stack Decisions, Component Mappings & Execution Blueprint • **Cognizant Hackathon Use Case 13**

Architecture Objective: Deliver an enterprise-ready, closed-loop AI platform running on a **100% free-tier stack** (Python + FastAPI + SQLite + scikit-learn + React + Vite + Leaflet) with Groq as the only external AI service, backed by a complete offline fallback engine for zero-dependency portability.

1. Technical Stack Decisions & Justifications

Component	PRD Baseline	TelConnect Production Choice	Technical Rationale
Database	PostgreSQL	SQLite (tci.db with WAL)	Zero setup on judge laptops; single-file portability; fast transactional throughput.
Vector Store	ChromaDB / Embeddings	TF-IDF Vector Space (scikit-learn)	Zero 500MB model download; sub-second cold starts; identical cosine similarity at demo scale.
Customer Agent	Procedural script	LangGraph Multi-Task StateGraph	6-node state machine orchestrating translation, routing, context, tools, and synthesis.
Multilingual	Regex replacement	Hugging Face DistilBERT + Groq	distilbert-base-multilingual-cased tokenization + Devanagari script enforcement.
Line Diagnostics	None (External)	Dynamic Line Telemetry Engine	Simulates download/upload speeds, ping, jitter, and loss tied to live incidents.
Voice Stack	None	Groq Whisper STT + Web Speech TTS	Bilingual voice transcription and audio readout (en-IN / hi-IN) directly in browser.
Admin & UI	Streamlit	React 18 + Vite SPA	Modern 40/60 split customer chat, Leaflet interactive map, and Recharts analytics.
Auth & Security	PyJWT + Bcrypt	hashlib.scrypt + HMAC Tokens	Standard library crypto; zero external security vulnerabilities; strict role gating.
AI Provider	Paid LLMs	Groq API (Llama-3 & Whisper)	High-speed inference (>300 t/s) with deterministic template fallback for offline demo.

2. Layered Codebase Architecture (backend/ & frontend/)

backend/ (FastAPI Server, Port 8000)

- app/db.py [Layer 4]: SQLite schema (users, complaints, incidents, notifications, chat_messages, kb_docs, feedback, audit_logs)
- app/auth.py [Layer 0]: Scrypt hashing, HMAC bearer tokens, role guards (Customer vs Admin)
- app/etl.py [Layers 1-2]: CSV upload, auto schema-mapping, data cleaning, PII redaction, Hinglish normalizer
- app/ml.py [Layer 3]: TF-IDF + LogReg classifier (Macro-F1 $\geq 80\%$), sentiment lexicon, urgency, priority scoring formula
- app/translator.py [Multilingual]: Hugging Face DistilBERT tokenizer, EN $\leftrightarrow$ HI translation, Devanagari enforcement
- app/agent_graph.py [Layer 5B]: LangGraph StateGraph agent, dynamic line diagnostics, ticket management tools
- app/assistant.py [Layer 5B]: Assistant endpoint, conversation message persistence, session state manager
- app/groq_client.py [AI Engine]: Groq API client (Llama-3 & Whisper) + deterministic offline fallback engine
- app/rag.py [Layer 5B]: Knowledge Base TF-IDF cosine similarity vector retrieval engine
- app/incidents.py [Layers 6-7]: Rolling-baseline spike detector, geospatial aggregator, AI root-cause dossier generator
- app/notify.py [Layer 8]: Affected-customer geo/service matching, proactive notification drafting & approval queue
- app/seed.py [Demo Data]: Synthetic 1,300 complaint generator with injected Raj Nagar broadband spike & demo accounts
- app/main.py [REST API]: Role-gated FastAPI routes + background daemon scheduler loop

frontend/ (React 18 + Vite SPA, Port 5173)

- src/pages/Landing.jsx: Animated radar hero landing page with feature showcase
- src/pages/Login.jsx: Role-separated tabbed authentication (Admin / Customer) & customer signup
- src/pages/Chat.jsx: Modern 40/60 Split Customer Portal (Profile, Status, Quick Actions, STT Mic, TTS, Diagnostic Cards)
- src/pages/Overview.jsx, Queue.jsx, Heatmap.jsx, Incidents.jsx, Alerts.jsx, Notify.jsx, Upload.jsx: Full operations control plane.

3. Demo Dataset & Outage Injection

The synthetic seed generator (app/seed.py) provisions **1,300 realistic telecom complaints** across 12 major Indian circles (Ghaziabad, Gurgaon, South Delhi, Mumbai, Bengaluru, Pune, Hyderabad, Chennai, Kolkata, Ahmedabad, Jaipur, Chandigarh).

- **Injected Outage Event:** An acute **4x broadband complaint spike in Raj Nagar, Ghaziabad** within the last 6 hours guarantees that on first launch: (1) The Leaflet Heatmap immediately shows a pulsating red spike over Raj Nagar; (2) The Spike Detector auto-triggers incident INC-xxx; (3) The Root-Cause Investigator synthesizes an evidence dossier; (4) The Notification Queue drafts targeted alerts for Raj Nagar broadband users.

4. Automated Testing & Quality Verification

The test suite runs under pytest with **100% pass rate across 49 automated test cases**:

- test_e2e.py (41 tests): Auth boundaries, CSV upload & schema mapping, PII redaction, ML Macro-F1 ($\geq 80\%$), SLA calculations, spike detection on injected Raj Nagar outage, root-cause evidence synthesis, notification approval, and CSV export.
- test_multilingual_agent.py (8 tests): Language detection, DistilBERT tokenization, English/Hindi translation engine, line diagnostic telemetry API, LangGraph state machine execution, Hindi chat flow, and Whisper voice endpoint validation.